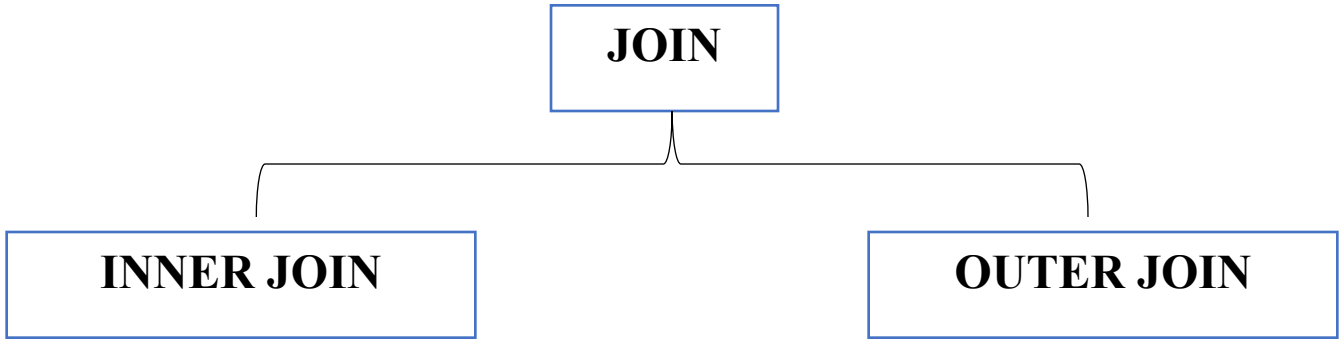
CSE3001
Database Management Systems

Modules	Syllabus
Module-3	**Relational Database design:** features, atomic domain. **SQL:** data definition, aggregate function, Null Values, nested sub queries, Joined relations. Triggers. Data Manipulation Language (DML) and Transaction Control Language (TCL) , Basic SELECT statements , Table Joins, Restricting and Sorting Data - Order By / Group By. Retrieving Data Using the SQL SELECT Statement, Single-Row Functions, Conversion Functions and Conditional Expressions, Reporting Aggregated Data Using the Group Functions, Displaying Data from Multiple Tables. Joins, Set Operators, DML Statements, Data Definition Language - Data Dictionary **Views -** Creating Sequences, Synonyms, and Indexes, Creating Views - Introduction to views, data independence, security, updates on views, comparison between tables and views.

SQL Joins:

- SQL Join is used to fetch data from two or more tables, which is joined to appear as single set of data.
- It is used for combining column from two or more tables by using values common to both tables.
- Join Keyword is used in SQL queries for joining two or more tables.
- Minimum required condition for joining table, is **(n-1)** where **n**, is number of tables.
- A table can also join to itself, which is known as, **Self Join**.

Types of JOIN:



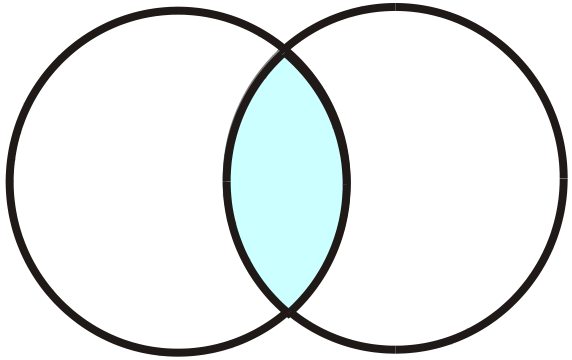
(Outcome contains only matching tuples)

- Theta Join
- Equi Join
- Natural Join
- Self Join

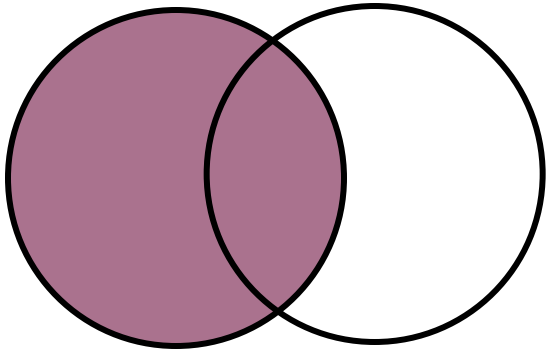
(Outcome contains all tuples from all tables)

- Left Join
- Right Join
- Full Join

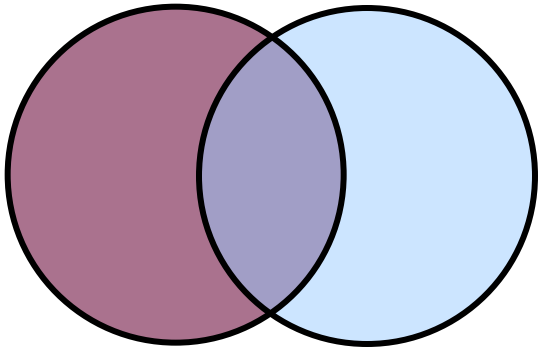
Types of JOIN:



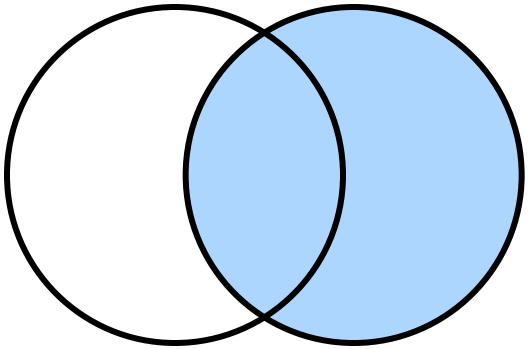
Inner Join



Left



Full



Right

INNER Join or EQUI Join

• This is a simple JOIN in which the result is based on matched data as per the equality condition specified in the SQL query.

•The syntax for Inner Join is,

```
SELECT column-name-list FROM
table-name1 INNER JOIN table-name2 on
table-name1.column-name = table-name2.column-name;
```

INNER Join or EQUI Join Example

Table1: class

ID	NAME
1	Abhi
2	Adam
3	Alex
4	Anu

Table2: class_info

ID	ADDRESS
1	Delhi
2	Mumbai
3	Chennai

```
SELECT * from class INNER JOIN class_info on class.id =
class_info.id;
```

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
3	Alex	3	Chennai

Natural JOIN

• Natural Join is a type of Inner join which is based on column having same name and same datatype present in both the tables to be joined.

•The syntax for Natural Join is,

```
SELECT * FROM table-name1 NATURAL JOIN table-name2;
```

Natural JOIN Example

Table1: class

ID	NAME
1	Abhi
2	Adam
3	Alex
4	Anu

Table2: class_info

ID	ADDRESS
1	Delhi
2	Mumbai
3	Chennai

```
SELECT * from class NATURAL JOIN class_info
```

ID	NAME	ADDRESS
1	Abhi	Delhi
2	Adam	Mumbai
3	Alex	Chennai

Inner Join – Example

loan

loan-number	*branch-name*	*amount*
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

borrower

customer-name	*loan-number*
Jones	L-170
Smith	L-230
Hayes	L-155

loan ⋈ *Borrower*

loan-number	*branch-name*	*amount*	*customer-name*
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

Outer JOIN

- Outer Join is based on both matched and unmatched data.
- Outer Joins subdivide further into,

Left Outer Join

Right Outer Join

Full Outer Join

LEFT Outer Join

• The left outer join returns a result set table with the matched data from the two tables and then the remaining rows of the left table and null from the right table's columns.

•Syntax for Left Outer Join is,

```
SELECT column-name-list FROM
table-name1 LEFT OUTER JOIN table-name2
ON table-name1.column-name = table-name2.column-name;
```

LEFT Outer Join Example-1

Table1: class

ID	NAME
1	Abhi
2	Adam
3	Alex
4	Anu
5	Ashish

Table2: class_info

ID	ADDRESS
1	Delhi
2	Mumbai
3	Chennai
7	Noida
8	Panipat

```
SELECT * FROM class LEFT OUTER JOIN class_info ON (class.id = class_info.id);
```

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
3	Alex	3	Chennai
4	Anu	Null	Null
5	Ashish	Null	Null

Left Outer Join – Example-2

loan

loan-number	*branch-name*	*amount*
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

borrower

customer-name	*loan-number*
Jones	L-170
Smith	L-230
Hayes	L-155

Left Outer Join

loan ⟕ *Borrower*

loan-number	*branch-name*	*amount*	*customer-name*
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*

RIGHT Outer Join

• The right outer join returns a resultset table with the matched data from the two tables being joined, then the remaining rows of the right table and null for the remaining left table's columns.

•Syntax for Right Outer Join is,

```
SELECT column-name-list FROM
table-name1 RIGHT OUTER JOIN table-name2
ON table-name1.column-name = table-name2.column-name;
```

RIGHT Outer Join Example-1

Table1: class

ID	NAME
1	Abhi
2	Adam
3	Alex
4	Anu
5	Ashish

Table2: class_info

ID	ADDRESS
1	Delhi
2	Mumbai
3	Chennai
7	Noida
8	Panipat

```
SELECT * FROM class RIGHT OUTER JOIN class_info ON (class.id = class_info.id);
```

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
3	Alex	3	Chennai
Null	Null	7	Noida
Null	Null	8	Panipat

Right Outer Join – Example-2

loan

loan-number	*branch-name*	*amount*
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

borrower

customer-name	*loan-number*
Jones	L-170
Smith	L-230
Hayes	L-155

- **Right Outer Join**

  loan ⟖ *borrower*

loan-number	*branch-name*	*amount*	*customer-name*
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	*null*	*null*	Hayes

FULL Outer Join

• The full outer join returns a result set table with the matched data of two table then remaining rows of both left table and then the right table.

•Syntax of Full Outer Join is,

```
SELECT column-name-list FROM
table-name1 FULL OUTER JOIN table-name2
ON table-name1.column-name = table-name2.column-name;
```

FULL Outer Join Example-1

Table1: class

ID	NAME
1	Abhi
2	Adam
3	Alex
4	Anu
5	Ashish

Table2: class_info

ID	ADDRESS
1	Delhi
2	Mumbai
3	Chennai
7	Noida
8	Panipat

```
SELECT * FROM class
FULL OUTER JOIN
class_info ON
class.id = class_info.id);
```

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
3	Alex	3	Chennai
4	Anu	Null	Null
5	Ashish	Null	Null
Null	Null	7	Noida
Null	Null	8	Panipat

Full Outer Join – Example-2

loan

loan-number	*branch-name*	*amount*
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

borrower

customer-name	*loan-number*
Jones	L-170
Smith	L-230
Hayes	L-155

Full Outer Join

$loan \mathbin{⟗} borrower$

loan-number	*branch-name*	*amount*	*customer-name*
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*
L-155	*null*	*null*	Hayes

Aggregate Functions:

List of Aggregate Functions in SQL:

SUM	-to calculate sum of numbers
MAX	-to find maximum value
MIN	- to find minimum value
AVG	-to calculate average of numbers
COUNT	-to count not null values in an attribute

Aggregate Functions:

Let`s consider a relation RESULT

SQL> select *from result;

RNO	CLGNAME	SEM	MARKS	NAME
1	OIST	I	71	Ajay
2	OIST	II	73	Aman
3	OCT	I	75	Bhanu
4	OIST	I	72	Ajit
5	OCM	I	76	Kartik
6	OCM	II	82	Karan
7	OCT	II	79	Ravi
8	OCT	I	80	Rakesh

Aggregate Functions:

```
SQL> select max(marks) from result;
MAX(MARKS)
----------
        82


SQL> select min(marks) from result;
MIN(MARKS)
----------
        71


SQL> select sum(marks) from result;
SUM(MARKS)
----------
       608
```

Aggregate Functions:

```
SQL> select avg(marks) from result;
AVG(MARKS)
----------
        76


SQL> select count(marks) from result;
COUNT(MARKS)
------------
           8


SQL> select rno,name from result where marks in(select max(marks) from result);
       RNO NAME
---------- --------------------
         6 Karan
```

Group by Clause:

Q. Find the maximum marks of OIST college.

```
SQL> select max(marks) from result where clgname='OIST';

MAX(MARKS)
----------
        73
```

Q. Find the maximum marks of each college.

```
SQL> select max(marks) from result group by clgname;
MAX(MARKS)
----------
        73
        82
        80
```

Group by Clause:

Q. Display the name of all the colleges with their maximum marks.

SQL> select max(marks),clgname from result group by clgname;

```
MAX(MARKS) CLGNAME
---------- ----------
        73 OIST
        82 OCM
        80 OCT
```

Q. Display the name of all the colleges with their maximum marks in each semester.

SQL> select max(marks),clgname,sem from result group by clgname,sem;

```
MAX(MARKS) CLGNAME    SEM
---------- ---------- ---
        79 OCT        II
        73 OIST       II
        82 OCM         II
        80 OCT        I
        76 OCM         I
        72 OIST       I
```

Group by Clause:

Q. Find the college name with their maximum marks but only those colleges, having average marks greater than 74.

SQL> select max(marks),clgname from result group by clgname having avg(marks)>74;

```
MAX(MARKS) CLGNAME
---------- ----------
        82 OCM
        80 OCT
```

Q. Find maximum marks of all colleges those are having at least 3 records.

SQL> select max(marks),clgname from result group by clgname having count(clgname)>2;

```
MAX(MARKS) CLGNAME
---------- ----------
        73 OIST
        80 OCT
```

Order by Clause:

Q. Display the maximum marks of each college in ascending order.

SQL> select max(marks),clgname from result group by clgname order by clgname;

```
MAX(MARKS)     CLGNAME
---------- ----------    ---
        82          OCM
        80          OCT
        73          OIST
```

Q. Find the maximum marks of all colleges with their name & semester in ordered list.

SQL> select max(marks),clgname,sem from result group by clgname,sem order by clgname;

```
MAX(MARKS) CLGNAME    SEM
---------- ---------- ---
        76          OCM       I
        82          OCM       II
        80          OCT       I
        79          OCT       II
        72          OIST      I
        73          OIST      II
```

Order by Clause:

Q. Find the maximum marks of all colleges with their name in ascending but semester in descending order.

SQL> select max(marks),clgname,sem from result group by clgname,sem order by clgname asc, sem desc;

MAX(MARKS)	CLGNAME	SEM
82	OCM	II
76	OCM	I
79	OCT	II
80	OCT	I
73	OIST	II
72	OIST	I

Complex Queries:

Consider relations:

Emp (eid, ename, dno)

Dept (dno, dname)

Sal (eid, salary)

SQL> create table emp (eid number(10) primary key, ename char(20), dno varchar2(10) references dept);

SQL> create table dept (dno varchar2(10) primary key, dname char(20));

SQL> create table sal(eid number(10) references emp, salary number(10), primary key(eid));

Complex Queries:

SQL> select *from emp;

EID	ENAME	DNO
1	Ajay	D01
2	Ajit	D02
3	Amit	D02
4	Aman	D03
5	Aman	D03
6	Anay	D01
7	Anant	D02

Complex Queries:

SQL> select *from dept;

DNO	DNAME
D01	Admin
D02	Finance
D03	HR

SQL> select *from sal;

EID	SALARY
1	40000
2	35000
3	35000
4	32000
5	47000
6	43000
7	52000

Complex Queries:

Q. Find those employee who works in HR Department.

Q. Display name of employees starts with ‘A’.

Q. Display those name that contains ‘n’ somewhere.

Q. Find details of those employees whose salary is more than 40000.

Q. Find the name of employee who earns minimum salary.

Q. Find department name that has highest salary.

Q. Find the average salary of each department.

Q. Find those department name that have at least 3 employees.

Q. Display the employee name in order but department in descending order.

Complex Queries:

Emp (empname, street, city)
Works (empname , compname, salary)
Company (compname, street, city)

Q. Find all employees who earn more salary than every employee of ‘ABC Cooperation’.

Q. Find name and street address of those employee who works in HCL.

Q. Find all company names located in every city in which ‘ABC Cooperation’ is located

Complex Queries:

emp1 (ssn, name)
sal1 (ssn, salary) // ssn referencing to emp
works1 (project#, ssn) // project# referencing to project & ssn referencing to emp
proj1 (project#, project_name, location)

Q. Display the projects name at Delhi.

SQL> select pname from proj1 where location='Delhi';

PNAME

HR

Complex Queries:

Q. Find the project name of employees whose salary is greater than 50000.

SQL> select pname from proj1 where proj1.project# in(select works1.project# from works1 where works1.ssn in(select sal1.ssn from sal1 where salry>50000));

PNAME

Finance

Account

SQL> select pname from sal1,works1,proj1 where salry>50000 and proj1.project# =works1.project# and works1.ssn=sal1.ssn;

PNAME

Finance

Account

Complex Queries:

Q. Retrieve the name and ssn of those employees working on project no ‘P101’.

```
SQL> select emp1.ssn, emp1.name from emp1 where emp1.ssn in(select works1.ssn
from works1 where project#='P01');
        SSN NAME
---------- --------------------
         1 Amit
         4 Sumit

SQL> select emp1.ssn, emp1.name from emp1,works1 where emp1.ssn=works1.ssn
and project#='P01';

       SSN NAME
---------- --------------------
         1 Amit
         4 Sumit
```

Complex Queries:

Q. Find the employees name working on project no P201 and salary>45000.

SQL> select emp1.name from emp1 where emp1.ssn in(select works1.ssn from works1 where project#='P201' and works1.ssn in(select sal1.ssn from sal1 where salary>45000));

SQL> select emp1.name from emp1,works1,sal1 where emp1.ssn=works1.ssn andworks1.ssn=sal1.ssn and project#='P201' and salary>45000;

Q. Find the employees name with maximum salary.

SQL> select emp1.name from emp1 where emp1.ssn in(select sal1.ssn from sal1 where salary in(select max(salary) from sal1));

SQL> select max(salary) from emp1,sal1 where emp1.ssn=sal1.ssn;

Modification of the Database

- The content of the database may be modified using the following operations:
  - Deletion
  - Insertion
  - Updating

- All these operations are expressed using the assignment operator.

Deletion:

- A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database.

- Can delete only whole tuples; cannot delete values on only particular attributes

- A deletion is expressed in relational algebra by:

$$r \leftarrow r - E$$

  where r is a relation and E is a relational algebra query.

Deletion Examples

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

- Delete all account records in the Perryridge branch.

$$account \leftarrow account - \sigma_{branch\text{-}name = \text{“Perryridge”}}(account)$$

- Delete all loan records with amount in the range of 0 to 50

$$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$$

Insertion

- To insert data into a relation, we either:
  - specify a tuple to be inserted
  - write a query whose result is a set of tuples to be inserted

- in relational algebra, an insertion is expressed by:

$$r \leftarrow r \cup E$$

  where r is a relation and E is a relational algebra expression.

- The insertion of a single tuple is expressed by letting E be a constant relation containing one tuple.

Insertion Examples

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

- Insert information in the database specifying that Smith has $1200 in account A-973 at the Perryridge branch.

  - account $\leftarrow$ account $\cup$ {(“Perryridge”, A-973, 1200)}
  - depositor $\leftarrow$ depositor $\cup$ {(“Smith”, A-973)}

Updating

- A mechanism to change a value in a tuple without charging *all* values in the tuple

- Use the generalized projection operator to do this task

$$r \leftarrow \prod_{F1,\, F2,\, \ldots,\, Fl,} (r)$$

- Each F_i is either
  - the *i*th attribute of *r*, if the *i*th attribute is not updated, or,
  - if the attribute is to be updated F_i is an expression, involving only constants and the attributes of *r*, which gives the new value for the attribute

Update Examples

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

- Make interest payments by increasing all balances by 5 percent.

$$account \leftarrow \Pi_{AN,\ BN,\ BAL\ *\ 1.05}(account)$$

where AN, BN and BAL stand for account-number, branch-name and balance, respectively.

- Pay all accounts with balances over $10,000 6 percent interest

Update Examples

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

- Pay all accounts with balances over $10,000 6 percent interest and pay all others 5 percent

$$account \leftarrow \Pi_{AN,\ BN,\ BAL\ *\ 1.06}(\sigma_{BAL > 10000}(account)) \cup \Pi_{AN,\ BN,\ BAL\ *\ 1.05}(\sigma_{BAL \leq 10000}(account))$$

Views

- In some cases, it is not desirable for all users to see the entire logical model (i.e., all the actual relations stored in the database.)

- Consider a person who needs to know a customer's loan number but has no need to see the loan amount. This person should see a relation described, in the relational algebra, by

$$\Pi_{customer\text{-}name,\ loan\text{-}number}\,(borrower \bowtie loan)$$

- Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a **view**.

View Definition

- A view is defined using the **create view** statement which has the form

  create view *v* **as** <query expression>

  where <query expression> is any legal relational algebra query expression. The view name is represented by *v.*

- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.

- View definition is not the same as creating a new relation by evaluating the query expression
  - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

View Examples

- Consider the view (named *all-customer*) consisting of branches and their customers.

create view *all-customer* **as**

$$\Pi_{\textit{branch-name, customer-name}} (\textit{depositor} \bowtie \textit{account}) \cup \Pi_{\textit{branch-name, customer-name}} (\textit{borrower} \bowtie \textit{loan})$$

- We can find all customers of the Perryridge branch by writing:

$$\Pi_{\textit{customer-name}} (\sigma_{\textit{branch-name}} = \text{“Perryridge”} (\textit{all-customer}))$$

Updates Through View

- Consider the person who needs to see all loan data in the *loan* relation except *amount.* The view given to the person, *branch-loan,* is defined as:

create view *branch-loan* **as**

$$\Pi_{\textit{branch-name, loan-number}}\,(\textit{loan})$$

- Since we allow a view name to appear wherever a relation name is allowed, the person may write:

$$\textbf{\textit{branch-loan}} \leftarrow \textbf{\textit{branch-loan}} \cup \textbf{\{(“Perryridge”, L-37)\}}$$

Thank You